



TECHNICAL UNIVERSITY OF DENMARK

Exercise 4 - Nonlinear Registration

Date of submission: 20th of October, 2025

Submitted By:

Holger Max Fløe Lyng, S214776

Maja Reben Johannesen, S204287

Dorthea Nørregaard, S204295

Othilia Wagner, S214780

Philip Vestergaard-Laustsen, s196248

Table of Contents

Introduction	1
1 Task 1: Normalize the intensity values of both images	2
1.1 Normalization with maximum intensity	2
1.2 Robust normalization	4
2 Task 2: Create basis functions for nonlinear deformation	7
2.1 Defining utility functions	8
2.2 Calculating and plotting the basis functions	9
2.3 Pruning edge basis functions	11
3 Task 3: Manipulate the image locally with basis function	13
3.1 Implentation of the code	16
4 Task 4: Implement Gauss-Newton optimization	17
5 Task 5: Experiment with different numbers of Basis Functions	22
Conclusion	31
6 Appendix A	32
6.1 Appendix code exhibit A	32

Introduction

In this exercise, the tasks are to implement a nonlinear image registration using a sum-of-squared-differences (SSD) similarity measure and the Gauss-Newton optimization method. The aim of the exercise is to align two sagittal MRI slices from different subjects, allowing smooth local deformations represented by cubic B-spline basis functions.

The deformation field $\delta_d(\mathbf{x}, \mathbf{w}_d)$, is defined as a weighted sum of basis functions:

$$\delta_d(\mathbf{x}, \mathbf{w}_d) = \sum_{m=0}^{M-1} w_{d,m} \phi_m(\mathbf{x}), \quad (1)$$

where $\phi_m(\mathbf{x})$ are the cubic B-spline basis functions and $\mathbf{w}_d$ are the weights for direction d . This formulation ensures that deformations are smooth and continuous across the image.

The displacement field $\mathbf{y}_d(\mathbf{x}, \mathbf{w}_d) = \mathbf{x} + \delta_d(\mathbf{x}, \mathbf{w}_d)$ maps points from the moving image to our fixed image. The optimal weights $\mathbf{w}_d$ are adjusted step by step to minimize the SSD cost function:

$$E(\mathbf{w}) = \frac{1}{2} \sum_{\mathbf{x}} [F(\mathbf{x}) - M(\mathbf{y}(\mathbf{x}, \mathbf{w}))]^2, \quad (2)$$

here F and M denote the fixed and moving images.

In **Task 1**, the two images are normalized to make their intensity ranges comparable.

For **Task 2**, we construct the cubic B-spline basis functions that define the nonlinear deformation field. **Task 3** explores how individual basis functions locally affect the image when specific weights are changed. In **Task 4**, the Gauss–Newton method is implemented to adjust the weights step by step and minimize the registration error. Finally, **Task 5** examines how the number of basis functions influences the accuracy and smoothness of the deformation.

Disclaimer: Parts of the coding, troubleshooting, and text refinement were assisted by using AI tools (ChatGPT). The use of AI tools in this work was conducted in accordance with DTU’s Honor Code and the regulations on academic integrity. All analysis, interpretations, and conclusions are the authors’ own.

1 Task 1: Normalize the intensity values of both images

Since MRI images are not calibrated, the pixel intensities in the fixed and the moving image are not necessarily comparable. As a first attempt to normalize the intensity values, divide each image by its maximum intensity value (to bring both images in the range $[0, 1]$) and display both images, as well as their histograms, side by side. Are the intensity values now comparable between the two images?

Compute a *robust* version of the maximum intensity in each image by locating the intensity value in the histogram so that 99% of the image pixels have a lower intensity. For each image, indicate the location of the value you find in the histogram using a verticle line (see code snippet given in the introduction).

Now rescale the image intensities again, this time using the robust maximum value instead. (Intensities higher than the robust maximum should be clipped so that all intensities fall within the range $[0, 1]$ after intensity normalization.) Display both normalized images, as well as their histograms, side by side again. Decide which normalization method makes the image intensities more comparable, explain why, and perform the remainder of the exercise using the normalized images of your choice.

1.1 Normalization with maximum intensity

The purpose of this task is to determine the best way to normalize the image data so that the images become comparable. First the images are normalized by their maximum intensity value. The function **histo(data)** creates the histogram of the given dataset.

```
1 def histo(data):
2     value = np.median( data )
3
4     histogram, edges = np.histogram( data.ravel(), bins=100 )
5     binCenters = ( edges[ 1: ] + edges[ :-1 ] ) / 2
6     binWidth = edges[ 1 ] - edges[ 0 ]
7     return value, histogram, binCenters, binWidth
```

```
1 #First normalization
2 data_fixed = fixed/fixed.max()
```

```

3 data_moved = moving/moving.max()
4 fig, ax = plt.subplots( 1, 2 )
5 visualize_image( ax[0], data_fixed, cmap='gray' )
6 visualize_image( ax[1], data_moving, cmap='gray' )
7 ax[0].set_title("Fixed Image")
8 ax[1].set_title("Moving Image")
9
10 fix_value, fix_histo, fix_binCen, fix_width = histo(data_fixed)
11 mov_value, mov_histo, mov_binCen, mov_width = histo(data_moved)
12
13 fig, ax = plt.subplots(1, 2, figsize=(10, 4))  # Create 1 row, 2 columns
14 ↪ of plots
15
16 # Fixed image histogram
17 ax[0].bar(fix_binCen, fix_histo, width=fix_width, color='steelblue')
18 ax[0].set_ylim(0, fix_histo[1:].max())
19 ax[0].set_xlim(0, data_fixed.max())
20 ax[0].plot([fix_value]*2, ax[0].get_ylim(), 'r')
21 ax[0].grid()
22 ax[0].set_title("Fixed image histogram")
23
24 # Moved image histogram
25 ax[1].bar(mov_binCen, mov_histo, width=mov_width, color='orange')
26 ax[1].set_ylim(0, mov_histo[1:].max())
27 ax[1].set_xlim(0, data_moved.max())
28 ax[1].plot([mov_value]*2, ax[1].get_ylim(), 'r')
29 ax[1].grid()
30 ax[1].set_title("Moved image histogram")
31 plt.tight_layout()
32 plt.show()

```

Figure 1 shows the histogram of the fixed and moving images after normalization with the respective maximum intensity values. The red line shows the median. We see that around 40% of the moving image range is used on outliers and 20% in the fixed image. Figure 2 shows the fixed and moving image after normalization, the intensities are still not comparable and the moving image is generally darker than the fixed images, which the compression of the histogram (fig 1) also show.

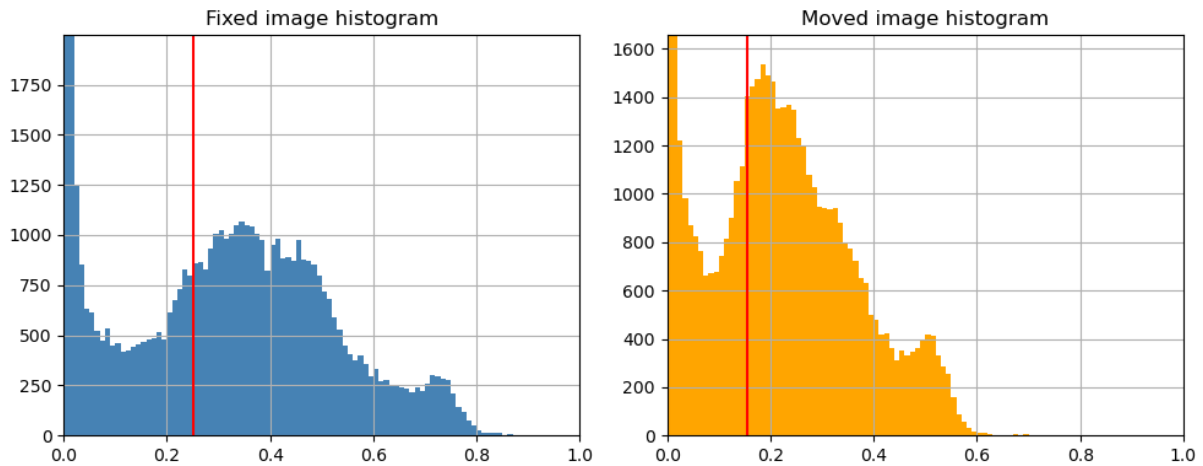


Figure 1: Intensity histograms after normalization, the red line line show the median.

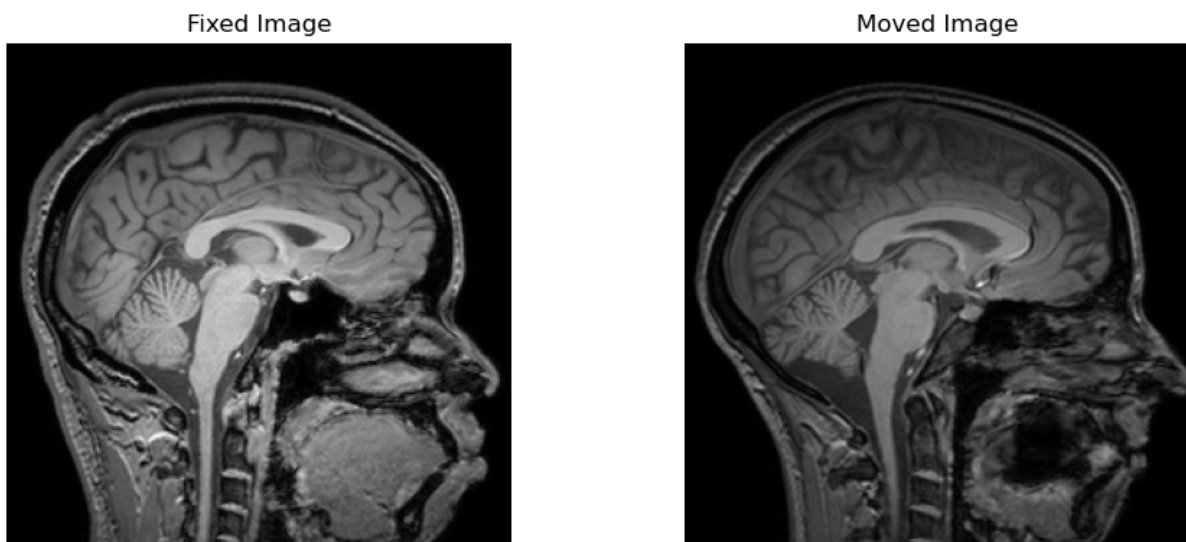


Figure 2: The two MRIs side by side, after normalization.

1.2 Robust normalization

Secondly, the data are normalized using a robust max, using the 99 percentile as the threshold, this ensures that data will be normalized without taking the outliers into account, since these will be assigned to the same value.

```

1 #Robust normalisation
2 fix_rm = np.percentile(fixed, 99)
3 mov_rm = np.percentile(moving, 99)

```

```

4  fixed_rm = np.where(fixed>fix_rm, fix_rm, fixed)
5  moved_rm = np.where(moving>mov_rm, mov_rm, moving)
6  fixed_rm = fixed_rm/fixed_rm.max()
7  moved_rm = moved_rm/moved_rm.max()
8
9
10 fix_value, fix_histo, fix_binCen, fix_width = histo(fixed_rm)
11 mov_value, mov_histo, mov_binCen, mov_width = histo(moved_rm)
12
13 # Plot fixed image histogram
14 fig, ax = plt.subplots(1, 2, figsize=(10, 4))
15 ax[0].bar(fix_binCen, fix_histo, width=fix_width, color='steelblue')
16 ax[0].set_ylim(0, fix_histo[1:].max())
17 ax[0].set_xlim(0, fixed_rm.max())
18 ax[0].plot([fix_value]*2, ax[0].get_ylim(), 'r')
19 ax[0].set_title("Fixed image histogram")
20 ax[0].grid()
21 # Plot moved image histogram
22 ax[1].bar(mov_binCen, mov_histo, width=mov_width, color='orange')
23 ax[1].set_ylim(0, mov_histo[1:].max())
24 ax[1].set_xlim(0, moved_rm.max())
25 ax[1].plot([mov_value]*2, ax[1].get_ylim(), 'r')
26 ax[1].set_title("Moved image histogram")
27 ax[1].grid()
28 plt.tight_layout()
29 plt.show()
30
31 ig, ax = plt.subplots( 1, 2 )
32 visualize_image( ax[0], fixed_rm, cmap='gray' )
33 visualize_image( ax[1], moving_rm, cmap='gray' )
34 ax[0].set_title("Fixed Image")
35 ax[1].set_title("Moving Image")

```

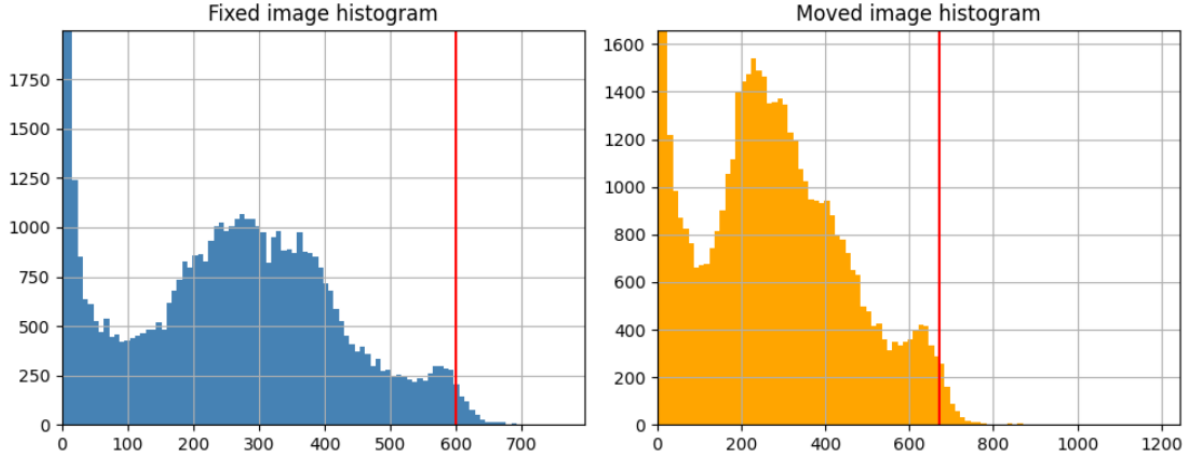


Figure 3: The histograms of the fixed and moving image before normalization, the red line indicates the 99th percentile, the data to the right of the line will be clipped.

Figure 3 shows the complete histogram before any normalization, the red line shows the 99th percentile of the fixed and moving image. It is the intensity values to the right of the red line that the robust normalized dataset will combine into the maximum bin.

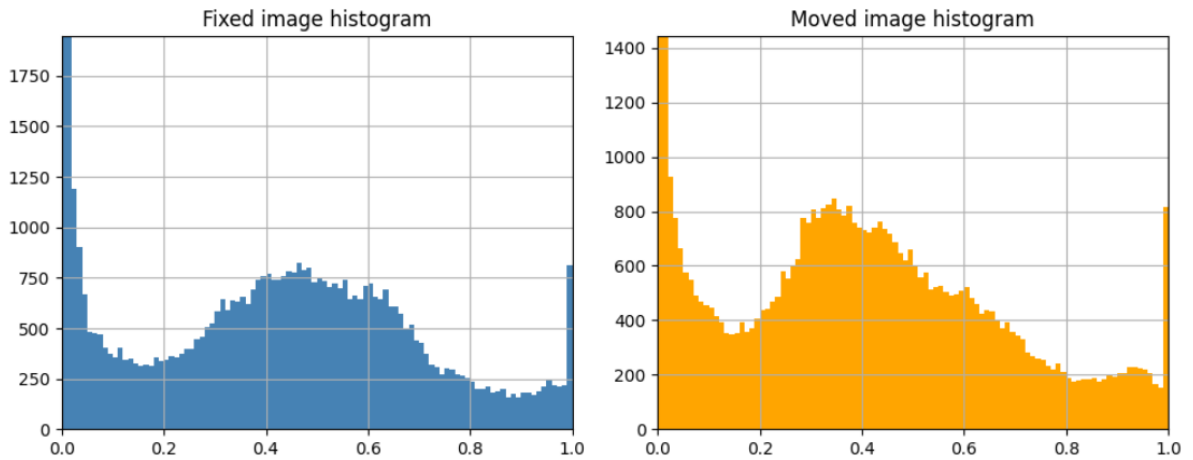


Figure 4: Histograms after robust intensity normalization. The fixed and moving image histograms after 99th-percentile clipping.

Figure 4 shows the histogram after the robust normalization, here we can see the histogram looks more alike than the previous normalized histogram (Fig 1), this indicates that the MRIs will be more comparable, which will give better registration results.

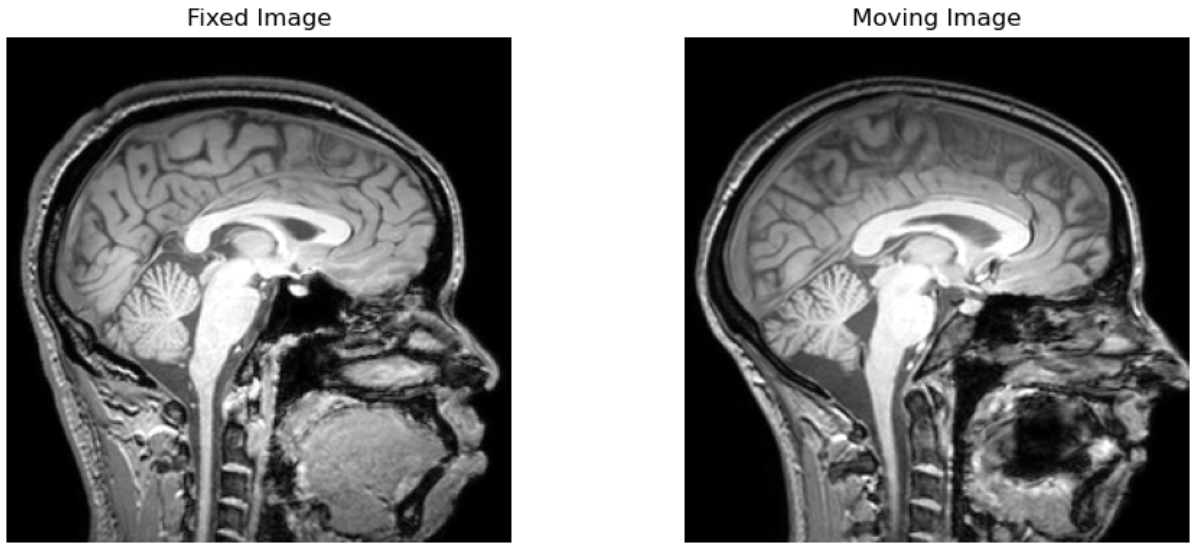


Figure 5: The two MRIs after the robust normalization where the 99 percentile have been clipped.

Figure 5 show the MRIs after the robust normalization, the two images are now comparable as predicted from the histograms. The images are more comparable because we map the 99% of the intensities into the whole range of $[0,1]$. Outliers are no longer dominating the normalization. This reduces their influence during registration, leading to more accurate results. Therefore, for the rest of the exercises we use the robust normalized data.

2 Task 2: Create basis functions for nonlinear deformation

In order to model the residual deformation in each direction $d = 1, \dots, D$, you are going to use a weighted set of basis functions:

$$\delta_d(\mathbf{x}, \mathbf{w}_d) = \sum_{m=0}^{M-1} w_{d,m} \phi_m(\mathbf{x}). \quad (3)$$

Here $\phi_m(\mathbf{x})$ are M basis functions, and $\mathbf{w}_d$ are the weights for direction d . For the purpose of this exercise, construct a set of $M = 7 \times 7 = 49$ separable, 2D cubic B-spline basis functions that cover the image area of the fixed image. Include a visualization of the basis functions, plotted in a 7×7 grid, in your report.

Hint:

- You should reuse the code for the 2D B-spline basis functions that you implemented for the exercise about image smoothing.

For the purpose of the exercise, retain only the central part of 5×5 basis functions, removing all the B-splines that are centered on one of the image edges. This will avoid strange-looking, large deformations of the image edges later on.

Hint:

- There are several ways of doing this. One option is to construct the 25 2D basis functions from only the five central 1D basis functions in each direction to start with:

```
np.kron(Phi2[:,1:-1], Phi1[:,1:-1])
```

instead of

```
np.kron(Phi2, Phi1).
```

2.1 Defining utility functions

In this task, the goal is to construct a set of B-spline basis functions, that we will later use to model nonlinear deformations. We begin by defining utility functions.

```
1 def interpolator(x, h, M, order):
2     #####
3     # Compute 1D B-spline basis functions of given order at points x
4     # with M basis functions and spacing h
5     #####
6     centers = h * np.arange(M)
7     phi1 = eval_BSpline((x[:, None] - centers[None, :])/h, order = order)
8     phi2 = eval_BSpline((x[:, None] - centers[None, :])/h, order = order)
9     return phi1, phi2
10
11 def plot_phi(M, phi_1, phi_2):
12     basis_2d = []
13     for i in range(M):
14         for j in range(M):
15             # Outer product makes an N x N 2D basis function
16             basis_2d.append(np.outer(phi_1[:, i], phi_2[:, j]))
17
```

```

18     basis_2d = np.array(basis_2d)    # shape = (49, N, N)
19
20     fig, axes = plt.subplots(M, M, figsize=(10, 10))
21     for i in range(M):
22         for j in range(M):
23             idx = i*M + j
24             axes[i, j].imshow(basis_2d[idx], cmap="viridis",
25                               ↪ origin="lower")
26             axes[i, j].axis("off")
27
28 def prune_edges(phi_1, phi_2):
29     return phi_1[:, 1:-1], phi_2[:, 1:-1]

```

2.2 Calculating and plotting the basis functions

Next, the 7×7 basis functions are computed, so that the grid spans the full image. It makes sure, that the deformation model covers the entire image domain.

```

1  M = 7
2  N = len(fixed_rm[1])
3  x = np.linspace(0, N-1, N)
4  h = (N-1)/(M-1)
5  phi_1, phi_2 = interpolator(x, h, M, 3)
6  phi = np.kron(phi_2, phi_1)
7
8  # Create 2D basis functions
9  plot_phi(M, phi_1, phi_2)
10
11 plt.suptitle("7x7 Cubic B-spline Basis Functions")
12 plt.tight_layout()
13 plt.show()

```

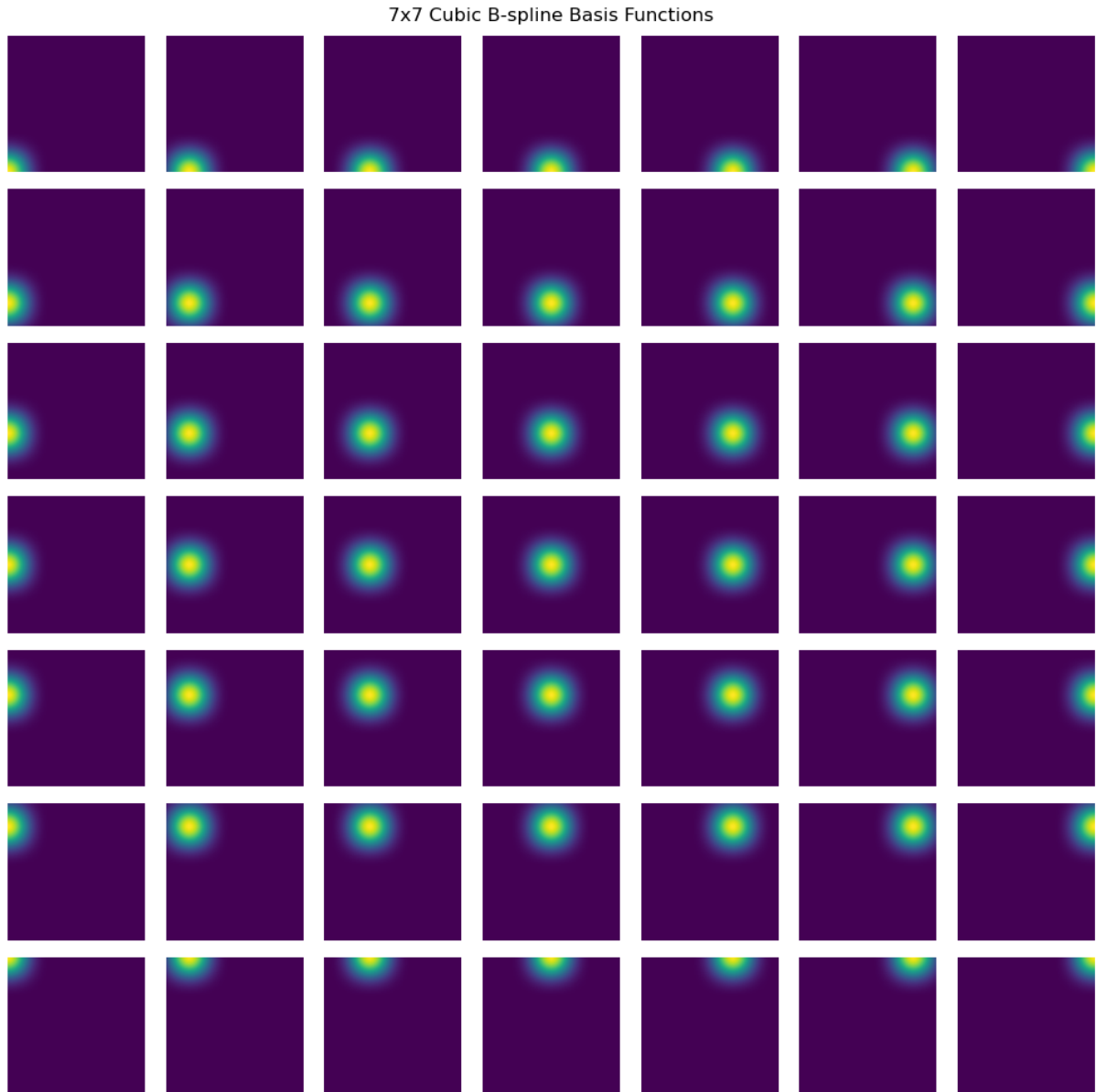


Figure 6: *7x7 cubic B-spline basis set. Visualization of the separable 2D cubic B-spline basis functions.*

Figure 6 shows the 7×7 set of cubic B-spline basis functions. Each of the functions influences a local region, and they together gives us a smooth, continuous basis for describing image deformations.

2.3 Pruning edge basis functions

It is important to prune the edges from the basis matrix to avoid edge "suck-in". The edge basis functions support outside of the image, which, during optimizing, can create unrealistic boarder warping. The outermost edges are constrained by fewer pixels than the centers and therefore only partially support the image. There are multiple ways to circumvent this; first pruning the edges, then using the Dirichlet boundary, a boundary regularizer, or padding.

```
1  phi1_central, phi2_central = prune_edges(phi_1, phi_2)
2
3  plot_phi(5, phi1_central, phi2_central)
4  phi_cent = np.kron(phi2_central, phi1_central)
5
6  plt.suptitle("5x5 Central Cubic B-spline Basis Functions")
7  plt.tight_layout()
8  plt.show()
9
```

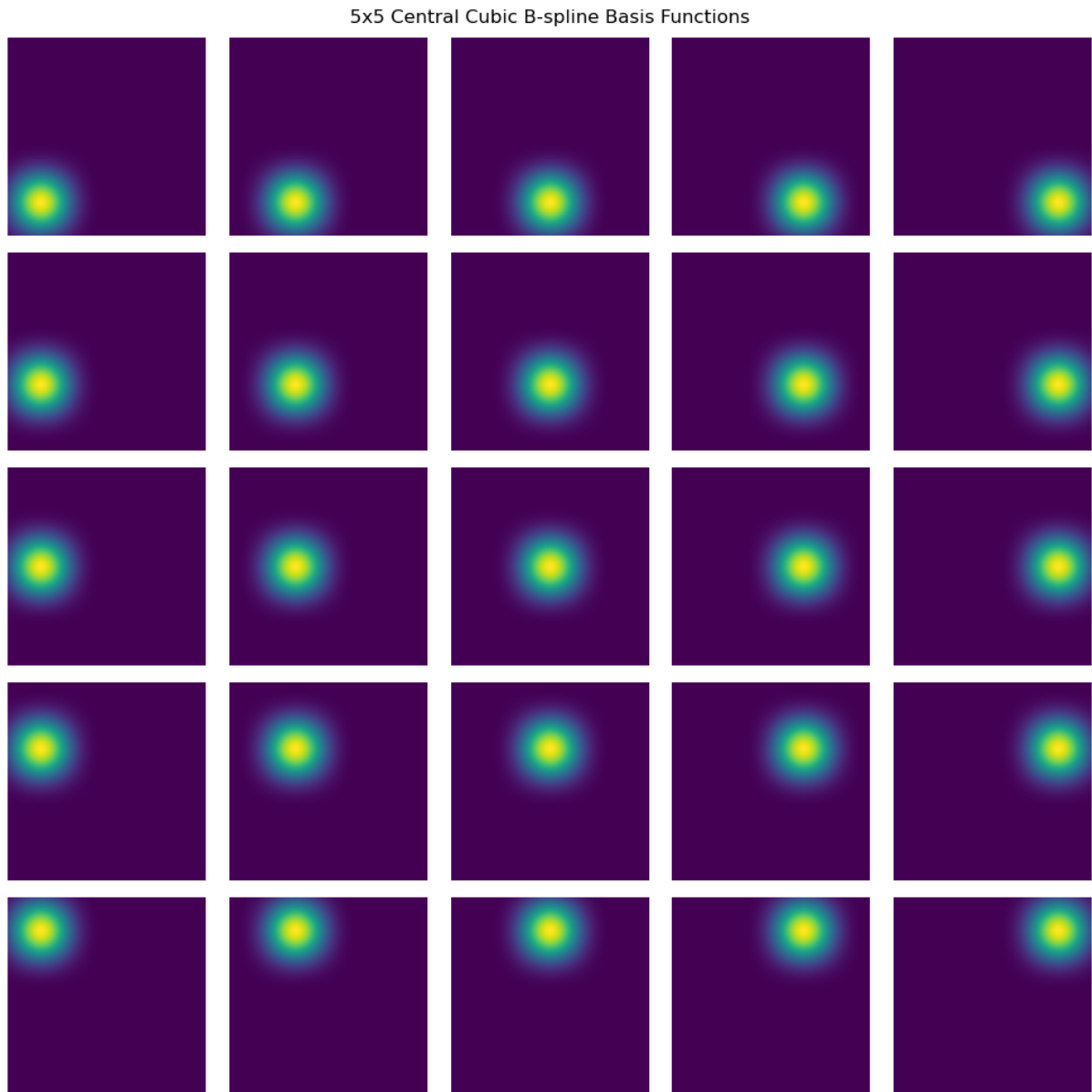


Figure 7: Central 5x5 basis subset.

The remaining 5×5 subset, seen in Figure 7, focuses on the central region of the image. This is where deformation is best defined. The reduced basis set will be used in the following tasks to model smooth, nonlinear transformations.

3 Task 3: Manipulate the image locally with basis function

In this task, you will use two weight vectors $\mathbf{w}_1$ and $\mathbf{w}_2$ to control the transformation model

$$y_d(\mathbf{x}, \mathbf{w}_d) = x_d + \delta_d(\mathbf{x}, \mathbf{w}_d), \quad d = 1, 2,$$

and visualize the effect of changing these weights on the resulting image deformation. For each choice of weights, you should show (1) how a regular image grid defined in the space of the fixed image looks after deformation, and (2) the corresponding interpolated moving image, resampled to the image grid of the fixed image.

In a first experiment, set all elements of $\mathbf{w}_2$ to zero, and set all elements *except one* in $\mathbf{w}_1$ to zero. Try a few different values for the one non-zero entry, show each time the resulting deformed image grid and the interpolated moving image, and comment on what you see.

In a second experiment, apply a non-zero value to the corresponding (i.e., controlling the same 2D basis function) element in $\mathbf{w}_2$ as well. Again visualize the result and comment.

We start by defining the utility functions needed to manipulate the images locally using the basis functions.

The function `make_grid()` takes an input image and creates the regular pixel grid in the image space. In the transformation model given in the task this corresponds to the x_d term giving the original coordinates.

The function `warp_image()` resamples an input image according to a displacement field. This corresponds to the $y_d(\mathbf{x}, \mathbf{w}_d)$ term in the transformation model as it maps the original pixels $\mathbf{x}$ to the warped pixels $\mathbf{y}$. This function also contains the calculations of the gradients that are needed in tasks 4 and 5.

The function `field_from_weights()` computes the deformation field from the weights and basis functions. This corresponds to the $\delta_d(\mathbf{x}, \mathbf{w}_d)$ term in the transformation model. It gives the displacement field, where the zero index gives the displacement in the x-direction and the first index gives the displacement in the y-direction.

```
1 def make_grid(image):  
2     #####
```

```

3         # compute a grid for visualization of the deformation
4         #####
5         yy, xx = np.meshgrid(
6             np.arange(image.shape[0]),
7             np.arange(image.shape[1]),
8             indexing= 'ij')
9         return yy, xx
10
11 def warp_image(img, disp , delta_step=None):
12     #####
13     # Warp an image according to a given displacement field using
14     #   ↪ cubic interpolation (3rd order)
15     # Optionally compute gradients of the warped image using finite
16     #   ↪ differences with step size delta_step
17     #####
18     H, W = img.shape
19     yy, xx = np.meshgrid(np.arange(H), np.arange(W), indexing="ij")
20     coords_y = yy + disp[1]    # disp[1] = dy
21     coords_x = xx + disp[0]    # disp[0] = dx
22
23     warped = map_coordinates(img, [coords_y, coords_x], order=3,
24                             ↪ mode="nearest")
25
26     if delta_step is not None: # optionally return gradients
27         M_y1_plus = map_coordinates(img, [coords_y + delta_step,
28             ↪ coords_x], order=3, mode="nearest")
29         M_y2_plus = map_coordinates(img, [coords_y, coords_x +
30             ↪ delta_step], order=3, mode="nearest")
31         g1 = (M_y1_plus - warped) / delta_step
32         g2 = (M_y2_plus - warped) / delta_step
33         return warped, g1, g2
34
35     return warped
36
37 def field_from_weights(phi, W):
38     #####
39     # compute the effect of the weights on the basisfunctions,
40     #   ↪ computing the deformation in x and y directions
41     #####
42     disp = phi @ W.T    # (65536, 25)
43     disp1 = disp.reshape(256, 256, 2).transpose(2,0,1) # reshape (65536,
44     ↪ 25) -> (25,256,256)
45     return disp1

```


As the first experiment we only changed the weight, w_1 . As we can see in Figure 8 the displacement of the grid is in the y-direction. When looking at the warped moving image instead, we can observe the displacement of the image is also in the y-direction. This can be an indication that w_1 is the displacement in the y-direction and not x-direction as we first thought.

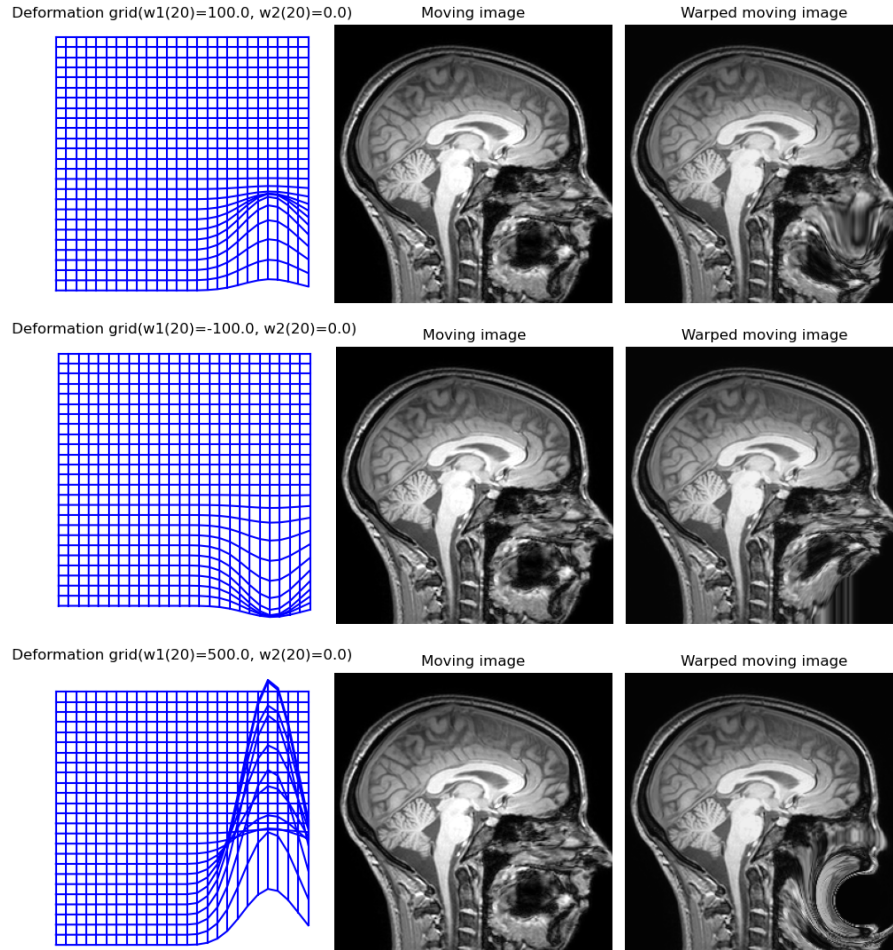


Figure 8: Experiment 1 with only manipulating the image in the y-direction.

For the second experiment we changed both w_1 and w_2 at the same index, meaning the same basis function. As we before only changed in the y-direction we can see in figure 9 that now when also changing w_2 the x-direction is also affected. This can also be observed in the warped moving image.

In this exercise, it is even more emphasized that w_1 affects the y-direction and w_2 affects the x-direction, in contrary to the first assumption.

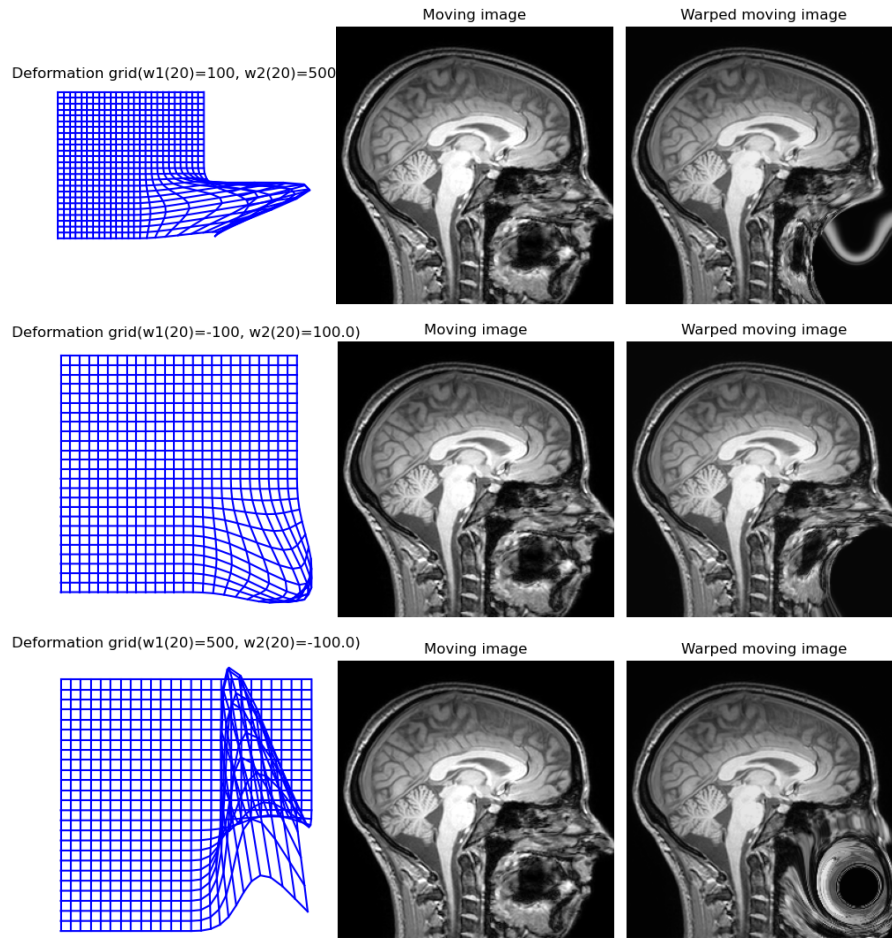


Figure 9: Experiment 2 with manipulating the image in the x - and y -direction.

3.1 Implentation of the code

Below, the functions described above are implemented within a for-loop. The loop iteratively substitutes the index (20) value with the i th value defined in *values*. During the first three iterations, changes occur only in the weights for the x -direction. After the third iteration, weight updates are applied in both the x - and y -directions, where the y -direction weights use the $i - 1$ th value.

```

1 w1 = np.zeros(len(phi_cent[1]))
2 w2 = np.zeros(len(phi_cent[1]))
3
4 idx = 20
5 values = [100.0, -100.0, 500.0, 100, -100, 500]
6

```

```

7  for i in range(len(values)):
8
9      W = np.vstack([w1[0:], w2[0:]]) # shape (2, 25)
10     W[0, idx] = values[i]
11     if i > 2:
12         W[1, idx] = values[i-1]
13
14     disp1 = field_from_weights(phi_cent, W)
15     warped = warp_image(moved_rm, disp1)
16     y, x = make_grid(moved_rm)
17     grid_x = x + disp1[0]
18     grid_y = y + disp1[1]
19
20     fig, ax = plt.subplots(1, 3, figsize=(10, 5))
21
22     visualize_grid(ax[0], grid_y, grid_x)
23     ax[0].set_title(f"Deformation grid(w1_{idx}={values[i]})")
24     ax[0].axis('off')
25
26
27     visualize_image(ax[1], moved_rm, cmap='gray')
28     ax[1].set_title('Moving image')
29     ax[1].axis('off')
30
31     visualize_image(ax[2], warped, cmap='gray')
32     ax[2].set_title('Warped moving image')
33     ax[2].axis('off')
34     plt.tight_layout()
35     plt.show()

```

4 Task 4: Implement Gauss-Newton optimization

To implement Gauss–Newton optimization, all previously defined utility functions are combined into a single function **registration**. This function defines one step of the optimization scheme.

First, it computes the deformation field, then warps the moving image and calculates the image gradients with respect to the warped image. Next, the residual τ and the energy are computed. The gradients are derived using the chain rule.

```

1 def registration(Phi, w, energy_history, F_flat, moving_rm,
  ↪ delta_step=0.001, lambda_LM=0.1):
2     M_basis = Phi.shape[1]
3     # Compute deformation
4     disp1 = field_from_weights(Phi, w) # disp shape (N_pixels, 2) and
  ↪ (2, H, W)
5
6     warped, g1, g2 = warp_image(moving_rm, disp1, delta_step) # also get
  ↪ gradients
7     g1 = g1.ravel() # gradient in x1 direction shape (N_pixels,)
8     g2 = g2.ravel() # gradient in x2 direction shape (N_pixels,)
9     M_warped = warped.ravel()
10
11     tau = F_flat - M_warped
12     energy = np.sum(tau**2)
13     energy_history.append(energy)

```

The *Jacobian* is approximated for each direction x and y , respectively. Approximating the Jacobian is standard when the analytical derivative is tedious, difficult, or unavailable. The Jacobian approximation begins inside **warp_image**, where the image gradients are computed. Ψ is constructed as: $\Psi = (G_1\Phi|\dots|G_D\Phi)$

```

1     # jacobian approximation (efficient elementwise multiplication)
  ↪ equation 2.12
2     Psi1 = Phi * g2[:, None]
3     Psi2 = Phi * g1[:, None]
4     Psi = np.hstack([Psi1, Psi2]) # shape (N_pixels, 2*M_basis)
5

```

In this implementation, the image gradients (g_1, g_2) are computed in **warp_image** using finite differences to approximate gradients, (g_1, g_2) . This computes a numerical approximation of the partial derivatives $\frac{\partial \mathcal{M}}{\partial y_1}$ and $\frac{\partial \mathcal{M}}{\partial y_2}$. Then, the Jacobian of the residual with respect to the deformation parameters (the weights $\mathbf{w}$) is constructed analytically using the chain rule:

$$\frac{\partial \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))}{\partial w_{d,m}} = \frac{\partial \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))}{\partial y_d} \cdot \frac{\partial y_d(\mathbf{x}_n, \mathbf{w})}{\partial w_{d,m}} = g_{d,n} \phi_m(\mathbf{x}_n).$$

Therefore, the approach combines both a numerical approximation of the image gradients and an analytical formulation of the Jacobian via the chain rule.

The next step is to calculate the update parameter ϵ according to Equation (2.15) in the lecture notes:

$$\epsilon = (\Psi^T \Psi + \lambda I)^{-1} \Psi^T \tau$$

Then, the weights are updated according to the parameter update rule from the lecture notes:

$$\mathbf{w} \leftarrow \mathbf{w} + \epsilon$$

This step is repeated until the parameters converge, or in this case, until iteration 100 is reached.

```

1  # Gauss-Newton update (with optional Levenberg-Marquardt
   ↪ regularization) equation 2.15
2  A = Psi.T @ Psi + lambda_LM * np.eye(2*M_basis)
3  b = Psi.T @ tau
4  eps = np.linalg.solve(A, b) #compute Epsilon
5
6  # Update weights
7  w += eps.reshape(2, M_basis)
8  return warped, w, energy_history, displ

```

Before beginning the optimization, it is necessary to define the run-time, delta step, basis functions, and weights. In this case, the Levenberg–Marquardt regularization term is set to zero, and an energy list is initialized to store the energy at each iteration.

It is important to note that the basis functions used here are the pruned basis functions from Task 2, as discussed there. This is a crucial step to remove the influence of edge basis functions.

```

1  t = 100 # number of Gauss-Newton iterations
2  delta_step = 0.001
3  Phi = phi_cent # Use central basis functions for deformation
4  lambda_LM = 0 # Levenberg-Marquardt regularization parameter
5  M_basis = Phi.shape[1]
6  # Initialize weights
7  w = np.zeros((2, M_basis))
8  energy_history = []
9
10 # Flatten coordinates for vectorized operations
11 F_flat = fixed_rm.ravel()

```

After initialization, the optimization scheme can begin. The **registration** function is called for each iteration and returns the warped image, the deformation field, the updated weights, and the energy for that iteration.

The warped image, deformation grid, and fixed image are displayed in the first iteration, every 20th iteration, and the final iteration.

The deformation in each direction is computed, and using Equation (2.7) in the lecture notes, the deformed grid, `grid_x` and `grid_y`, can be calculated from `displ`, which are then used for visualization.

```
1  for it in range(1, t+1):
2      warped, displ, w, energy_history = registration(Phi, w,
3          ↪ energy_history, F_flat, moved_rm, delta_step=0.001,
4          ↪ lambda_LM=lambda_LM)
5      # Visualize every 20th iteration (and first) using the requested
6      ↪ plotting style
7      if it == 1 or 20 == 0 or it == t:
8          y, x = make_grid(warped) # compute grid
9          grid_x = x + displ[0] # apply deformation to grid in x direction,
10             ↪ using equation 2.7
11          grid_y = y + displ[1] # apply deformation to grid in y direction,
12             ↪ using equation 2.7
13
14          fig, ax = plt.subplots(1, 3, figsize=(10, 5))
15          visualize_grid(ax[0], grid_x, grid_y)
16          ax[0].set_title(f"Deformation grid (iter {it})")
17
18          ax[1].set_title('Warped moving image')
19          ax[1].imshow(np.rot90(warped), cmap='gray')
20
21          ax[2].set_title('Fixed image')
22          ax[2].imshow(np.rot90(fixed_rm), cmap='gray')
23          plt.tight_layout()
24          plt.show()
25          print(warped.max())
26
27      # Plot energy curve
28      plt.figure()
29      plt.plot(np.arange(1, t+1), energy_history)
30      plt.xlabel('Iteration')
31      plt.ylabel('SSD Energy')
```

```
26 plt.title('Gauss-Newton Energy Curve')
27 plt.grid()
28 plt.show()
```

In Figure 10, the first and final iterations of the optimization scheme are shown. The deformation of the moving image is evident, especially in the neck, brain stem, corpus callosum, and mouth regions. Visually, the moving image in the final iteration is closely aligned with the fixed image compared with the first iteration. This observation is supported by the corresponding decrease in energy. However, even with a 5x5 basis function, some unnatural warping of the gyri and sulci is observed, among other areas, which is undesirable if structural or voxel-wise differences are the study subject.

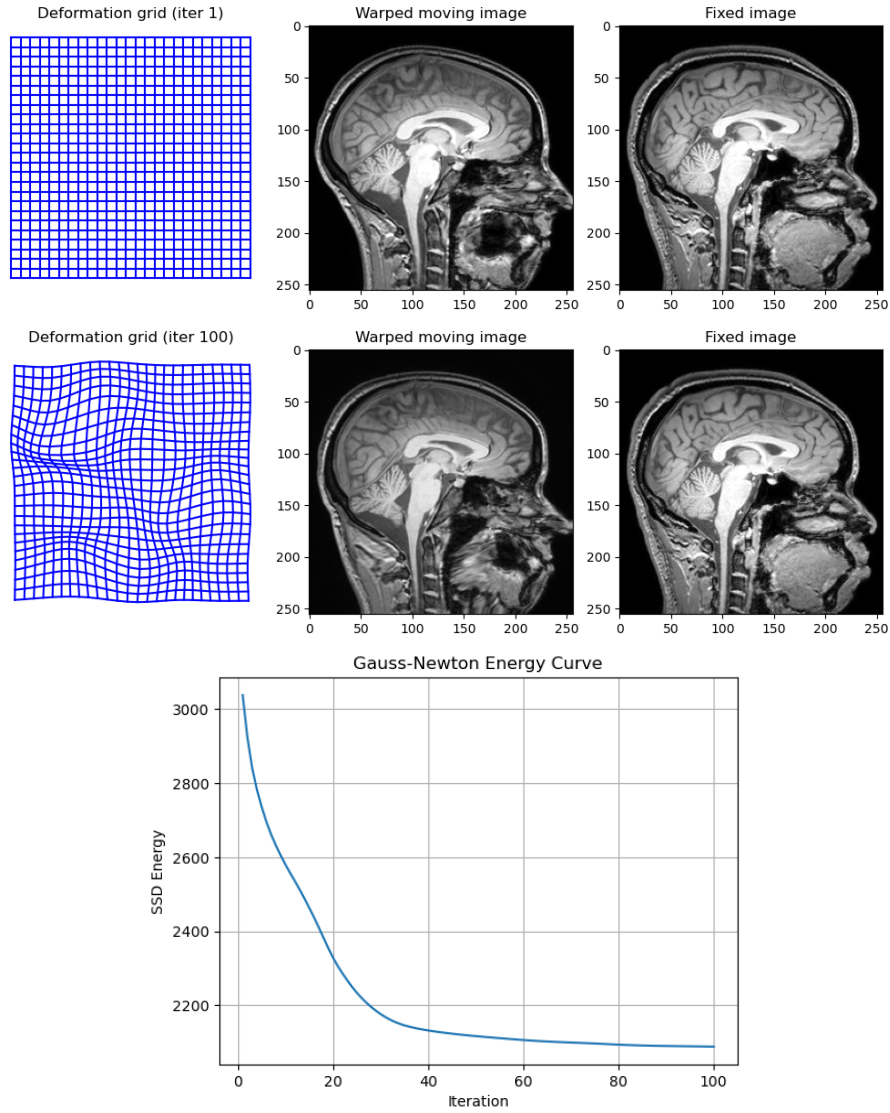


Figure 10: A) The plot of the first step before any updates to the weights. B) Plot of the final step. C) Plots the energy at each iteration

5 Task 5: Experiment with different numbers of Basis Functions

Gauss-Newton optimization has now been implemented into **registration**, which have made it easy to use and can be used to test the effect basis size has on the registration. In this task, three different basis function matrices will be tested, of sizes; 7x7, 5x5 and 3x3. The basis

functions are constructed in the same manner as done in task 2, e.g. construct a larger matrix than needed, then prune to wanted size. The Basis functions are created in the first for-loop.

```

1  # construct the 3 different basis function sets, through iteration
2  # 1. All basis functions sizes
3
4  M = np.array([9, 7, 5])
5  N = fixed_rm.shape[1]
6  x = np.linspace(0, N-1, N)
7  phi_all = []
8  for m in range(len(M)):
9      M_current = M[m]
10     h = (N-1)/(M_current-1)
11     phi_1, phi_2 = interpolator(x, h, M_current, 3) # Creates basis
        ↪ function.
12     phi_1, phi_2 = prune_edges(phi_1, phi_2) # Prune edges
13
14     phi_current = np.kron(phi_2, phi_1) # combine the two 1d basis
        ↪ function to a 2d
15     phi_all.append(phi_current)
16
17
18 t = 100 # number of Gauss-Newton iterations
19 delta_step = 0.001
20
21 # Flatten coordinates for vectorized operations
22 F_flat = fixed_rm.ravel()
23 m_basis = [7, 5, 3]
24 for basis in range(len(M)):
25     Phi = phi_all[basis]
26     # Initialize weights
27     M_basis = m_basis[basis]
28     w = np.zeros((2, M_basis*M_basis))
29     energy_history = []
30
31     for it in range(1, t+1):
32         warped, disp_show, w, energy_history = registration(Phi, w,
33             ↪ energy_history, F_flat, moved_rm, delta_step=0.001,
34             ↪ lambda_LM=0) # calculate w, compute energy and update w
35         # Visualize every 20th iteration (and first) using the requested
36             ↪ plotting style
37         if it == 1 or it == t:
38             y, x = make_grid(warped) # compute grid
39             grid_x = x + disp_show[0] # apply deformation to grid in x
40             ↪ direction

```

```

20     grid_y = y + disp_show[1] # apply deformation to grid in y
    ↪ direction
21
22     fig, ax = plt.subplots(1, 3, figsize=(10, 5))
23     visualize_grid(ax[0], grid_y, grid_x)
24     ax[0].set_title(f"Deformation grid {M_basis}x{M_basis} (iter
    ↪ {it})")
25
26     ax[1].set_title('Warped moving image')
27     ax[1].imshow(np.rot90(warped), cmap='gray')
28
29     ax[2].set_title('Fixed image')
30     ax[2].imshow(np.rot90(fixed_rm), cmap='gray')
31     plt.tight_layout()
32     plt.show()
33 # Plot energy curve
34     plt.figure()
35     plt.plot(np.arange(1, t+1), energy_history)
36     plt.xlabel('Iteration')
37     plt.ylabel('SSD Energy')
38     plt.title(f'Gauss-Newton Energy Curve basis function
    ↪ {M_basis}x{M_basis}')
39     plt.grid()
40     plt.show()
41     print(energy_history[99])
42

```

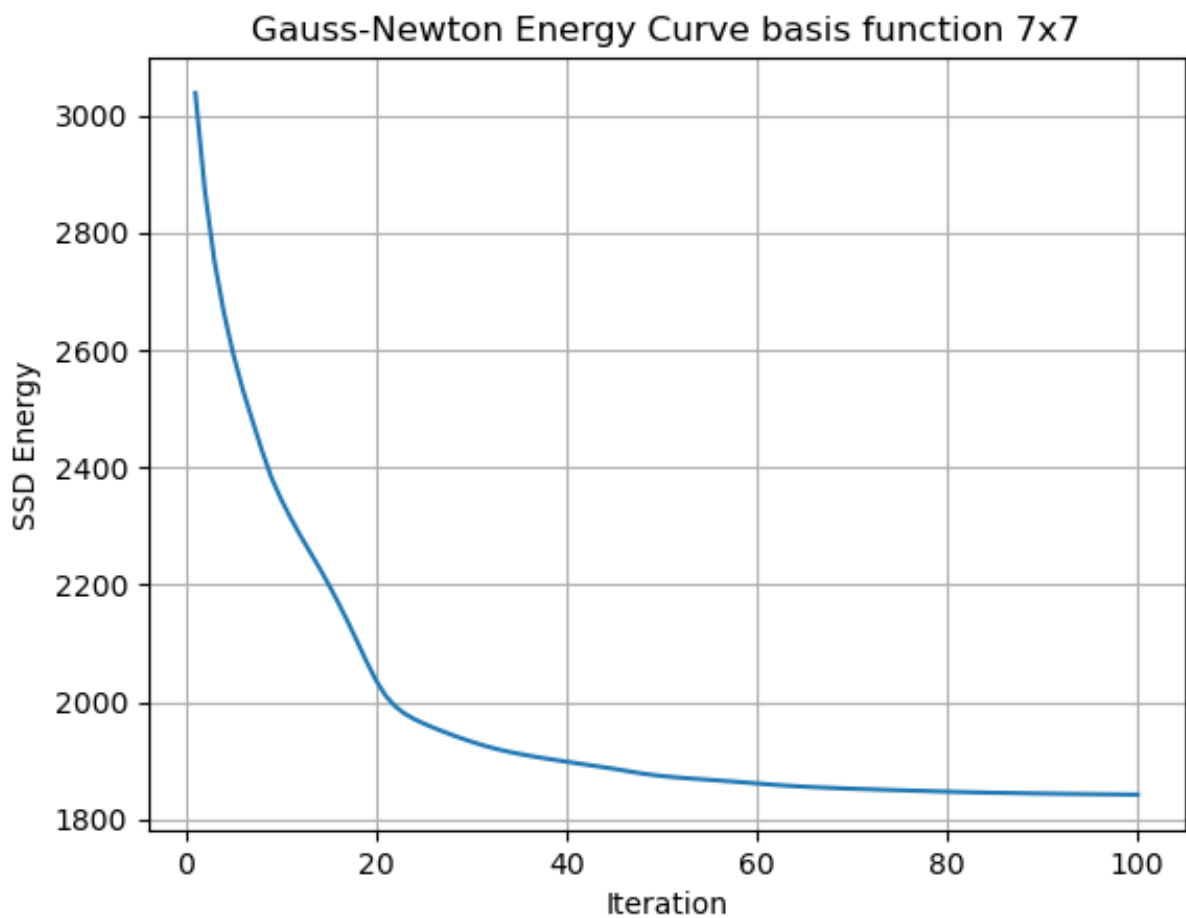
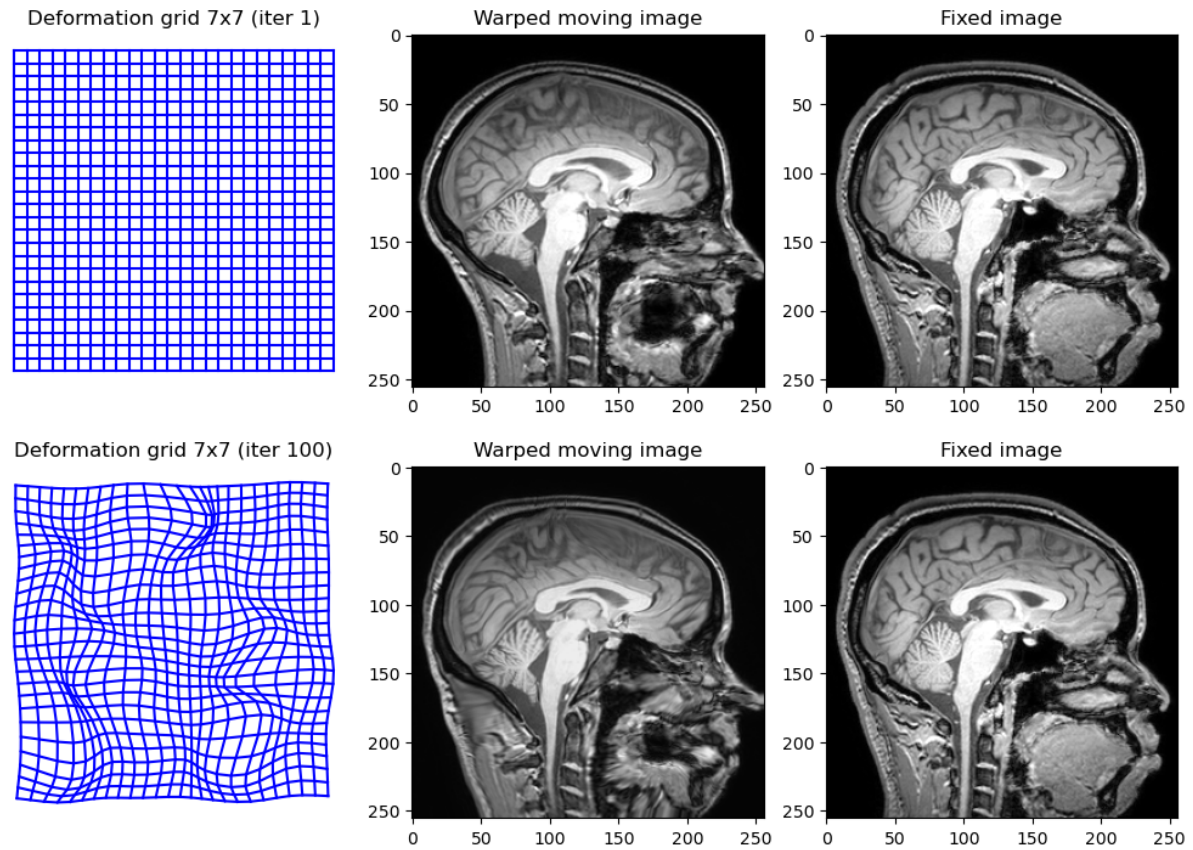


Figure 11: In this figure the results from simulation with 7x7 basis functions are shown. First the original deformed grid, moving image and fixed image are shown at the top. In the middle the deformed grid, warped moving image and fixed moving image, in the final iteration of Gauss-Newton optimization, are shown. In the bottom, the Energy curve from Gauss-Newton optimization with 7x7 basis functions are shown.

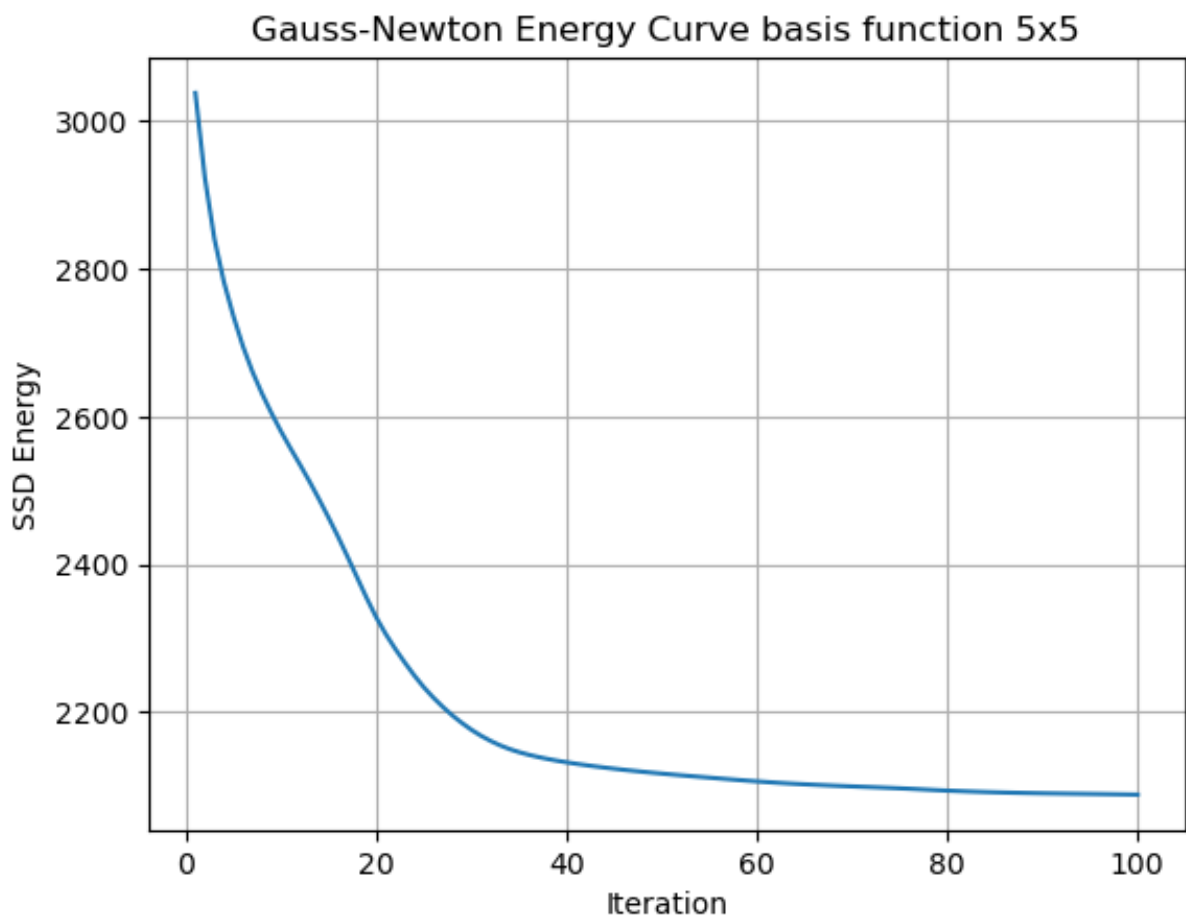
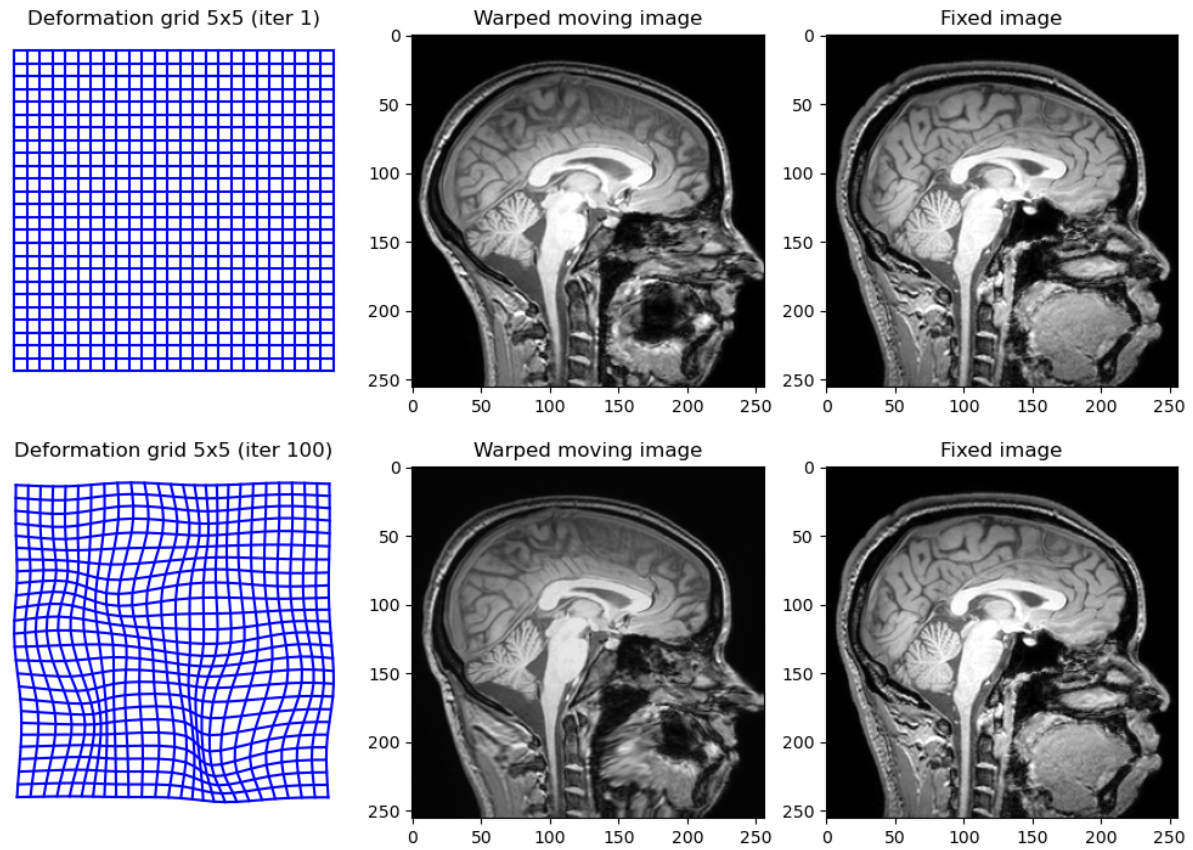


Figure 12: In this figure the results from simulation with 5x5 basis functions are shown. First the original deformed grid, moving image and fixed image are shown at the top. In the middle the deformed grid, warped moving image and fixed moving image, in the final iteration of Gauss-Newton optimization, are shown. In the bottom, the Energy curve from Gauss-Newton optimization with 5x5 basis functions are shown.

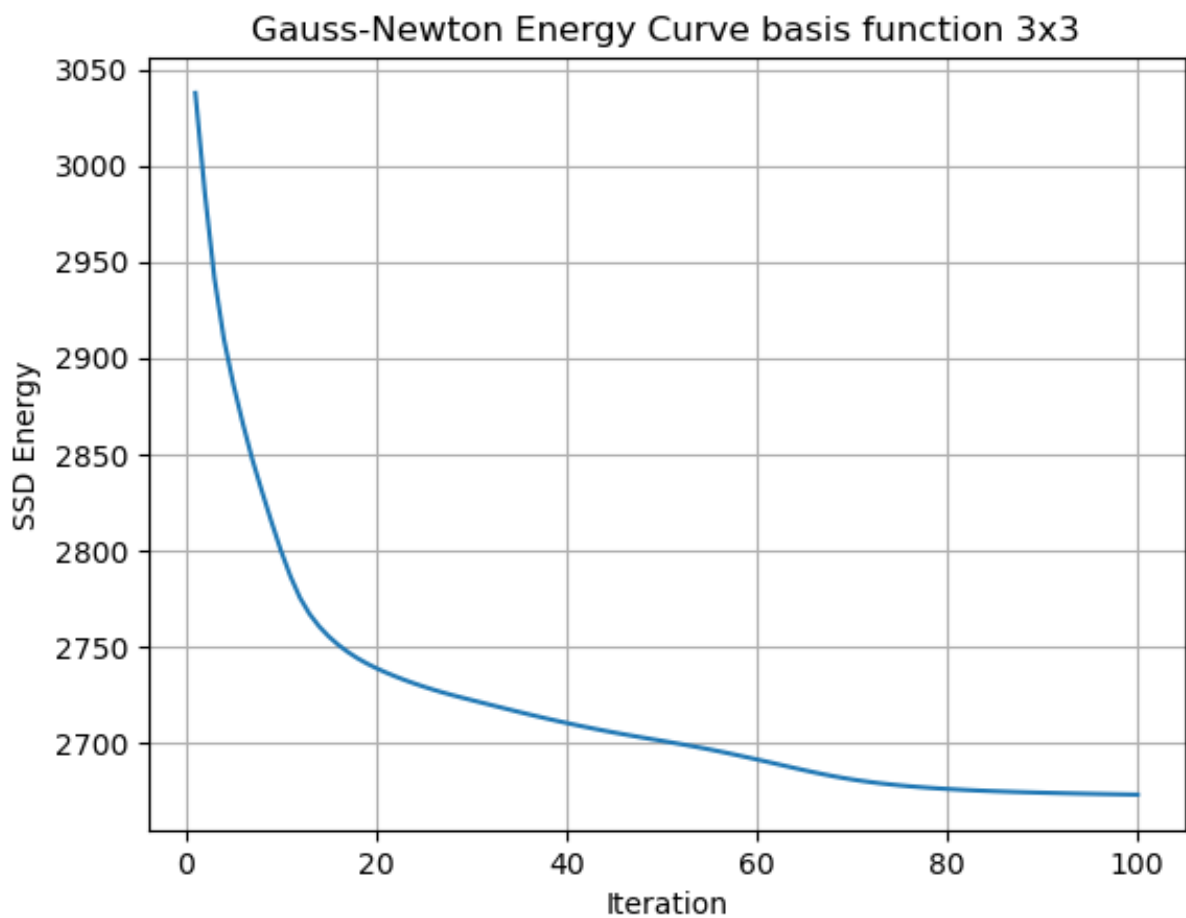
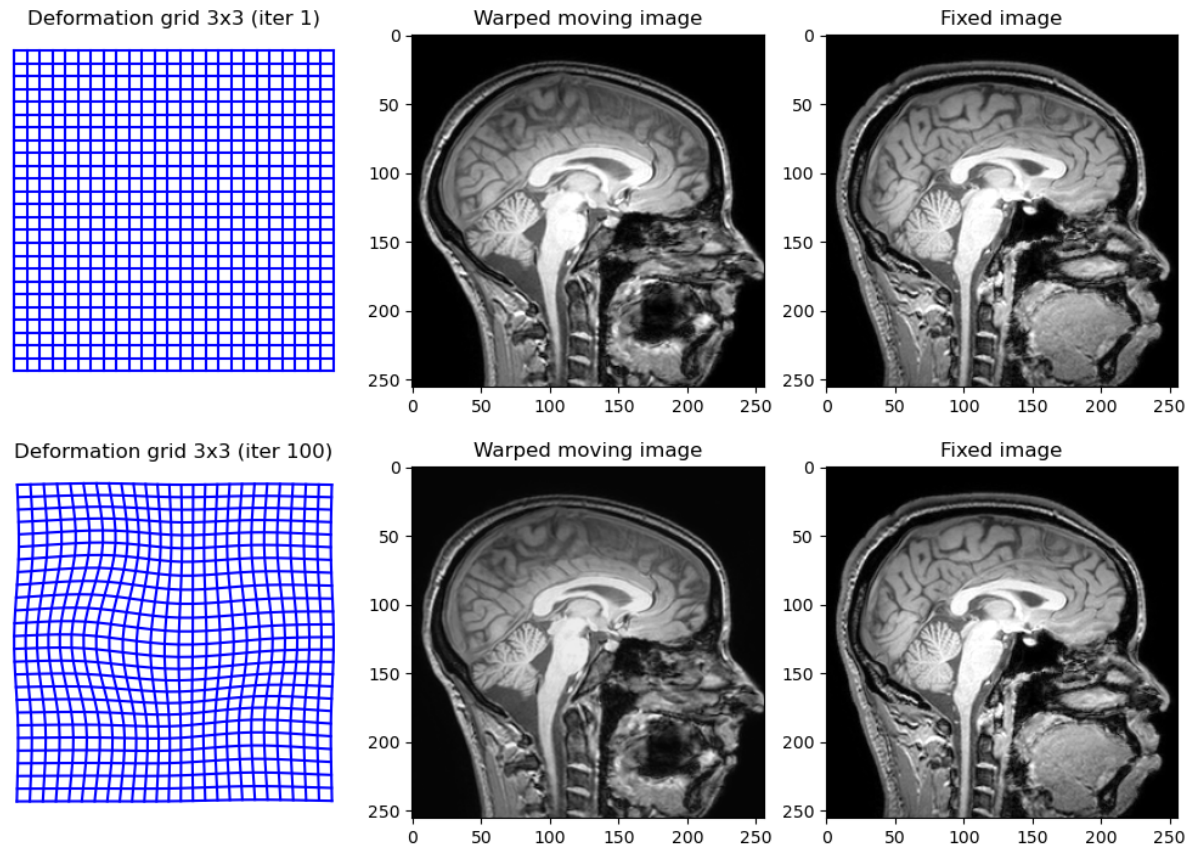


Figure 13: In this figure the results from simulation with 3x3 basis functions are shown. First the original deformed grid, moving image and fixed image are shown at the top. In the middle the deformed grid, warped moving image and fixed moving image, in the final iteration of Gauss-Newton optimization, are shown. In the bottom, the Energy curve from Gauss-Newton optimization with 3x3 basis functions are shown.

From figure 11, 12 and 13 we see that decreasing the number of basis functions, leads to increased energy, in the final step of the Gauss-Newton optimization algorithm. The energy is calculated as:

$$E(\mathbf{w}) = \sum_{n=1}^N [\mathcal{F}(\mathbf{x}_n) - \mathcal{M}(\mathbf{y}(\mathbf{x}_n, \mathbf{w}))]^2$$

Which is basically the summed squared difference between warped moving image and fixed image. Thereby, one would assume that a low energy, equals better fit. In these cases (fig. 11, 12 and 13), that does seem to be the case. Upon closer inspection, this does not hold however. We tried warping with 23x23 basis functions and when looking at figure 14 and 15, its apparent that the energy is much lower, but the image fit looks worse. Hence, even though the fit seems to be better *quantitatively*, *qualitatively* that is not the case. It seems like there is a trade-off between model flexibility (more basis), and qualitative fit. So in conclusion, optimizing number of basis, is also a task for the image analysis engineer, and there is such a thing as too many and too few basis.

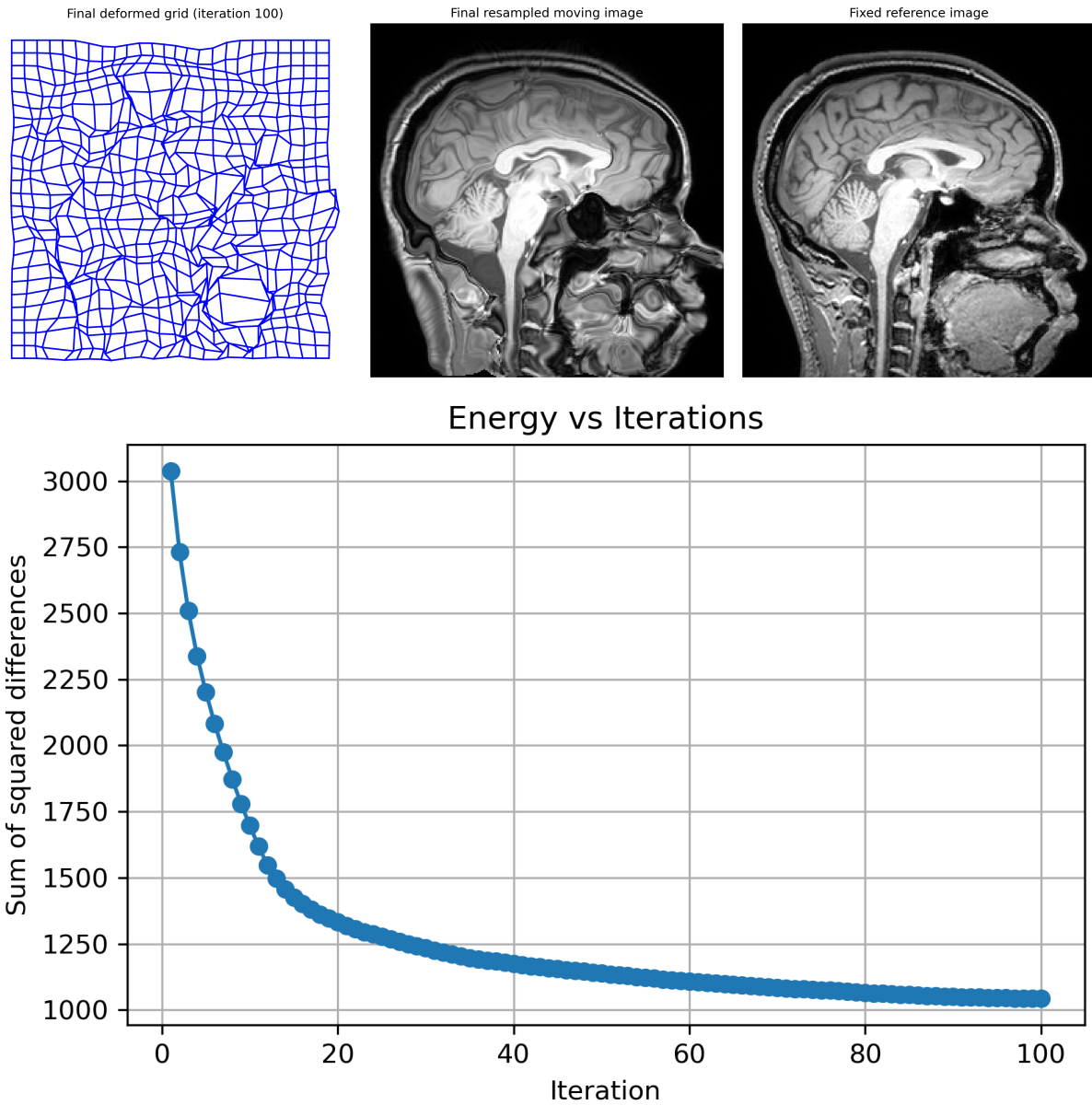


Figure 14: In this figure the results from simulation with 23x23 basis functions are shown. First the deformed grid, warped moving image and fixed moving image, in the final iteration of Gauss-Newton optimization, are shown. In the bottom, the Energy curve from Gauss-Newton optimization with 23x23 basis functions are shown.

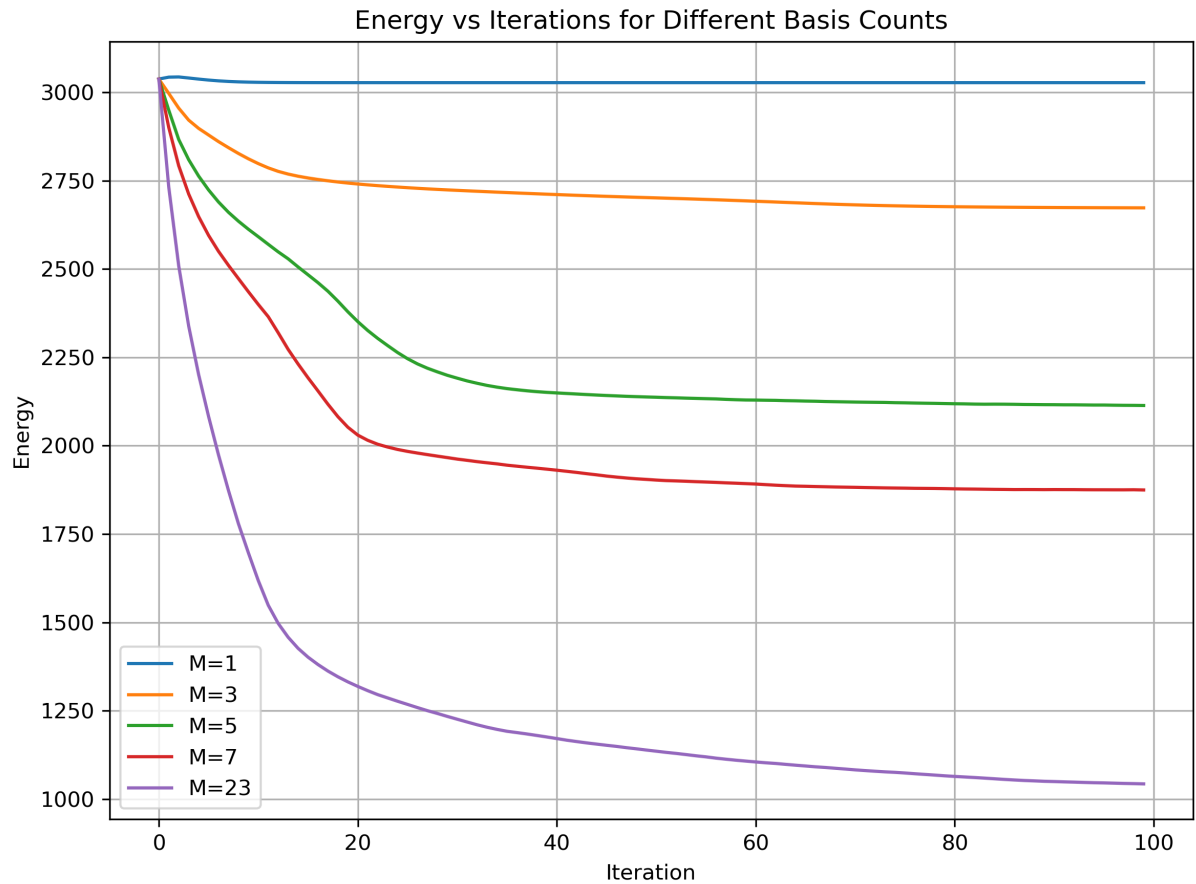


Figure 15: In this figure the results from simulations where the number of basis was varied and the energy recorded is shown.

NOTE:

Figure 14 and 15 were generated from code, which can be seen in appendix 6.1.

Conclusion

In this exercise, we implemented nonlinear image registration using cubic B-spline basis functions and the Gauss-Newton optimization method. By using robust intensity normalization, the two images were made comparable and we made the normalization less affected by outliers.

We created a set of 2D cubic B-spline basis functions and pruned the edge functions to avoid these unrealistic boundary deformations. When we made local manipulations, it showed that

changing the individual weights can produce smooth and predictable deformations. During optimization, the moving image gradually aligned with the fixed image. The SSD energy decreased across iterations, which confirmed that the method minimizes the registration errors.

Some small artifacts were still visible near the structural boundaries. They could be reduced by stronger regularization or if we used a similarity measure less sensitive to the intensity differences. Overall, the method showed how nonlinear registration enables smooth, flexible deformations in a way that affine or rigid transformations can't achieve.

During Task 3, we initially thought that w_1 described the deformation in the x-direction. However, as observed in Exercise 1 when analyzing the grid, w_1 actually influenced the deformation in the y-direction. This demonstrates that we cannot always rely on our initial assumptions.

During Task 5, we noticed that even though increasing the number of basis leads to lower energy, the fit does not generally improve qualitatively. This indicates a need for optimization of the number basis itself.

Through this exercise, we gained practical experience with nonlinear registration and a better understanding of how basis functions, normalization, and regularization affect image alignment.

6 Appendix A

6.1 Appendix code exhibit A

```
1  #We first make some functions for evaluation of the different spline  
   ↪ families:  
2  def B0(x):  
3      #Allocate space:  
4      B0x = np.zeros((len(x)))  
5      B0x = np.zeros_like(x, dtype=float)  
6  
7      # First interval  
8      B0x[(-0.5 < x) & (x < 0.5)] = 1.0  
9  
10     # Boundary  
11     B0x[np.isclose(np.abs(x), 0.5)] = 0.5  
12  
13     return B0x  
14  
15  def B1(x):  
16      #Allocate space:  
17      B1x = np.zeros((len(x)))  
18      B1x = np.zeros_like(x, dtype=float)  
19  
20      #  
21      idx = np.abs(x)<1  
22      B1x[idx] = 1.0-np.abs(x[idx])  
23  
24      return B1x  
25  
26  def B3(x):  
27      #Allocate space:  
28      B3x = np.zeros((len(x)))  
29      B3x = np.zeros_like(x, dtype=float)  
30  
31      #  
32      idx = np.abs(x)<1 #Indices where it is true for the first interval,  
        ↪ could also have used np.where i think.  
33      B3x[idx] = (2/3)-np.abs(x[idx])**2+(np.abs(x[idx])**3)/2  
34      idx = (1 <= np.abs(x)) & (np.abs(x)<2)  
35      B3x[idx] = ((2-np.abs(x[idx]))**3)/6  
36  
37      return B3x  
38
```

```

39 def eval_BSpline(x, order=0):
40     """
41     Evaluates the uniform B-spline of order "order" at the locations in
42     ↪ vector "x"
43     """
44     # First it seems we need to construct some basis functions, one for
45     ↪ each case. And then use an if statement to switch between what is
46     ↪ used.
47     if order == 0:
48         return B0(x)
49     elif order == 1:
50         return B1(x)
51     elif order == 3:
52         return B3(x)
53     else:
54         raise ValueError("Order not implemented")
55
56 #And the Phi matrix.
57 def GeneratePhiMatrix(x, order, N, M, h):
58     phi = np.zeros((N,M))
59
60     for m in range(M):
61         phi[:,m] = eval_BSpline((x-m*h)/h), order)
62
63     #PhiProducts = phi.T@phi
64     #InverseProduct = np.linalg.solve(PhiProducts,phi.T)
65     #S = phi@InverseProduct
66     return phi #We return a tuple with the matrices. Nvm i changed it.
67
68 #And now the one for 2D.
69 def GeneratePhi2D(M1, M2, N1, N2, order):
70     locs1 = np.arange(N1)
71     h1 = (N1-1)/(M1-1)
72     Phi1 = GeneratePhiMatrix(locs1, order, N1, M1, h1)
73     #And the other basis
74     locs2 = np.arange(N2)
75     h2 = (N2-1)/(M2-1)
76     Phi2 = GeneratePhiMatrix(locs2, order, N2, M2, h2)
77
78     # In this case because N1=N2 and M1=M2 we dont care about direction,
79     ↪ and they are essentially idendical.
80     PHI_2d = np.kron(Phi2, Phi1)
81     return PHI_2d
82
83 def GeneratePhi2D_Wo_Edgeing(M1, M2, N1, N2, order):

```

```

80     locs1 = np.arange(N1)
81     h1 = (N1-1)/(M1-1)
82     Phi1 = GeneratePhiMatrix(locs1, order, N1, M1, h1)
83     #And the other basis
84     locs2 = np.arange(N2)
85     h2 = (N2-1)/(M2-1)
86     Phi2 = GeneratePhiMatrix(locs2, order, N2, M2, h2)
87
88     # In this case because N1=N2 and M1=M2 we dont care about direction,
89     ↪ and they are essentially identical.
90     #We use the hint from the task
91     PHI_2d = np.kron( Phi2[:,1:-1], Phi1[:,1:-1])
92     return PHI_2d
93
94 #We make a function that takes the basis function, and the weights, and
95 ↪ computes a grid.
96 def ComputeGrid(Basis, W1, W2, X1, X2):
97     W = np.column_stack([W1, W2])
98     delta_d = Basis@W
99     #Split deformation field into components and reshape to image grid
100    delta1 = delta_d[:, 0].reshape(N1, N2)
101    delta2 = delta_d[:, 1].reshape(N1, N2)
102    Y1 = X1+delta1
103    Y2 = X2+delta2
104    return Y1, Y2, delta1, delta2
105
106 #First, it might be a good idea to make a function that can atleast
107 ↪ calculate the interpolated moving image based on some W1, W2 vectors.
108 def Compute_Moving_Img_Deformed(Phi_2d, W1, W2, X1, X2):
109     #Compute grid
110     Y1, Y2, _, _ = ComputeGrid(Phi_2d, W1, W2, X1, X2)
111     #Flatten the deformed grid
112     coords = np.vstack([Y1.ravel(), Y2.ravel()]) # shape (2, N1*N2)
113     #Interpolate
114     deformed_moving = map_coordinates(Robust_moving, coords, order=3,
115     ↪ mode='constant', cval=0.0)
116     #Reshape back to image grid
117     deformed_moving = deformed_moving.reshape(N1, N2)
118     return deformed_moving
119
120 #And now we make a function for it.
121 def GaussNewtonOpti(delta, lambda_reg, iterations, M1, M2, order,
122     ↪ MovingImg, FixedImg, X1, X2):

```

```

120     energy = []
121
122     #Initialize the basis:
123     N1 = FixedImg.shape[0]
124     N2 = FixedImg.shape[1]
125     Phi_2d = GeneratePhi2D_Wo_Edgeing(M1, M2, N1, N1, order) #Both images
126     ↪ need same resolution if we wanna use this later
127
128     #Initialize weights
129     w1 = np.zeros(Phi_2d.shape[1])
130     w2 = np.zeros(Phi_2d.shape[1])
131
132     for i in range(1, iterations + 1):
133         #Compute deformed grid
134         Y1, Y2, __, __ = ComputeGrid(Phi_2d, w1, w2, X1, X2)
135
136         #Interpolate
137         coords = np.vstack([Y1.ravel(), Y2.ravel()])
138         M_warped = map_coordinates(MovingImg, coords, order=3,
139         ↪ mode='constant', cval=0.0).reshape(N1, N2)
140
141         #Compute Residuals
142         tau_n = FixedImg - M_warped
143
144         #Compute gradients
145         g1 = (map_coordinates(MovingImg, [Y1 + delta, Y2], order=3,
146         ↪ mode='constant') -
147         ↪ map_coordinates(MovingImg, [Y1, Y2], order=3,
148         ↪ mode='constant')) / delta
149         g2 = (map_coordinates(MovingImg, [Y1, Y2 + delta], order=3,
150         ↪ mode='constant') -
151         ↪ map_coordinates(MovingImg, [Y1, Y2], order=3,
152         ↪ mode='constant')) / delta
153
154         #Construct Psi matrix
155         Psi_x = g1.ravel()[ :, None] * Phi_2d
156         Psi_y = g2.ravel()[ :, None] * Phi_2d
157         Psi = np.hstack([Psi_x, Psi_y])
158
159         #Find Epsilon
160         tau = tau_n.ravel()
161         epsilon = np.linalg.inv(Psi.T @ Psi + lambda_reg *
162         ↪ np.eye(Psi.shape[1])) @ (Psi.T @ tau)

```

```

158     #Update weights
159     w1 += epsilon[:w1.size]
160     w2 += epsilon[w1.size:]
161
162     # Compute energy
163     E = np.sum(tau_n ** 2)
164     energy.append(E)
165
166     # Show it all
167     fig, ax = plt.subplots(1, 3, figsize=(15, 5))
168
169     #Deformed grid
170     visualize_grid(ax[0], Y1, Y2)
171     ax[0].set_title(f"Final deformed grid (iteration {iterations})")
172
173     #Deformed (resampled) moving image
174     visualize_image(ax[1], M_warped, cmap='gray', vmin=0, vmax=1)
175     ax[1].set_title("Final resampled moving image")
176     ax[1].axis('off')
177
178     #Fixed image
179     visualize_image(ax[2], FixedImg, cmap='gray', vmin=0, vmax=1)
180     ax[2].set_title("Fixed reference image")
181     ax[2].axis('off')
182
183     plt.tight_layout()
184     #Save that shit
185     plt.savefig(f'{M1-2}x{M1-2}_Grid_Moving_Fixed.png', dpi=300,
186               ↪ bbox_inches='tight')
187     plt.show()
188
189     # Energy curve
190     plt.figure(figsize=(6, 4))
191     plt.plot(range(1, iterations + 1), energy, '-o')
192     plt.title('Energy vs Iterations')
193     plt.xlabel('Iteration')
194     plt.ylabel('Sum of squared differences')
195     plt.grid(True)
196     plt.tight_layout()
197     #Also save
198     plt.savefig(f'{M1-2}x{M1-2}_Energy_Curve.png', dpi=300,
199               ↪ bbox_inches='tight')
200     plt.show()
201
202     print(f"Final energy: {energy[-1]:.6f}")

```

```

201
202     return w1, w2, energy
203
204 order1 = 3
205 #Test it.
206 W1, W2, Eng = GaussNewtonOpti(delta, lambda_reg, iterations, M1, M2,
    ↪ order1, Robust_moving, Robust_fixed, X1, X2)
207
208
209 #Now we do it for different numbers of basis functions.
210 order_5 = 3
211 basis_sizes = [3, 5, 7, 9, 25]
212 Lambda_5 = 0.1
213 Iterations_5 = 100
214 delta_5 = 0.001
215 W1_L = []
216 W2_L = []
217 ENG_L = []
218
219 for M in basis_sizes:
220     print('\n\n\n')
221     print(f"Running Gauss-Newton with M1=M2={M} basis functions")
222     print('\n\n')
223
224     #Run Gauss-Newton
225     W1, W2, Eng = GaussNewtonOpti(MovingImg=Robust_moving,
    ↪ FixedImg=Robust_fixed, X1=X1, X2=X2, M1=M, M2=M, delta=delta_5,
    ↪ lambda_reg=Lambda_5, iterations=Iterations_5, order=order_5)
226
227     #Store results in lists
228     W1_L.append(W1)
229     W2_L.append(W2)
230     ENG_L.append(Eng)
231
232
233
234 #Energy curves with all the tested basis.
235 plt.figure(figsize=(8, 6))
236 for i, M in enumerate(basis_sizes):
237     plt.plot(ENG_L[i], label=f"M={M}")
238 plt.title("Energy vs Iterations for Different Basis Counts")
239 plt.xlabel("Iteration")
240 plt.ylabel("Energy")
241 plt.legend()
242 plt.grid(True)

```

```
243 plt.tight_layout()  
244 plt.savefig(f'Final_Energy_All_plots', dpi=300, bbox_inches='tight')  
245 plt.show()
```